

decoupling_review

CONST-051(B) Decoupling Review — per-file decisions

Round 41 close-out¹⁶ — Task #255 first pass.

Per CONST-051(B), every owned-by-us submodule MUST remain fully decoupled from any specific consuming project. The Task #250 audit surfaced 7 owned submodules with HelixCode references in non- governance files. This document records the per-file analysis + remediation decision per the operator’s “per-file review needed” mandate.

Decision taxonomy

Class	Definition	Remediation
legitimate-cross-project	The reference is intentional cross-project context (e.g. a per-target generator file named after the target; a shared utility’s namespace prefix). The submodule is genuinely reusable; the reference is descriptive, not a hardcoded dependency.	Document and pass.
cosmetic-only	The reference is in a comment / docstring / description string that names the originating project. Behavior is generic; only the labelling is project-specific.	Genericise the comment in a follow-up.
genuine-violation	The reference is a hardcoded filesystem path, hostname, or asset name that makes the submodule unusable by a different consumer without source edits.	Refactor to config-injection (env var / config file / constructor parameter). Track as fine-grained task.

Per-submodule × per-file findings

Challenges (21 files)

The 21 files are all challenges/p1-fNN-*/run.sh and challenges/p2-fNN-*/run.sh phase-runner scripts. Each script names HelixCode-specific phase IDs (F01-F30) and exercises HelixCode-specific features.

Classification: legitimate-cross-project. The Challenges submodule is HelixCode's Challenge bank by design; per-Challenge run.sh scripts are HelixCode-specific artefacts in a Challenge bank that also hosts other projects' Challenges (under separate prefixes). The Challenge bank's structure is: one directory per (project, phase, feature) triple — "p1-f20-theme-system" means "project 1 (HelixCode), phase 1, feature 20 (theme system)". Other consuming projects add their own pX-fYY-* directories. CONST-051(B) is honoured at the per-Challenge-script level: a Challenge script that operates ON HelixCode is naturally named after HelixCode. The Challenges submodule as a whole IS reusable — by adding a new project's Challenge directory tree.

Action: none.

containers (2 files)

containers/scripts/load_api_keys.sh (1 file)

Two HelixCode references in comments: - Line 3: # HelixCode API-key loader: prefers \$HOME/api_keys.sh ... - Line 70: # ... ApiKey_OpenAI, etc., but the HelixCode providers would not find them

The script logic itself is generic. Function names use helixcode_* prefix as a namespace.

Classification: cosmetic-only (comments + function namespace).

Action: genericise the two comments in a future round (replace "HelixCode" with "downstream"/"generic"). Function names keep their helixcode_* prefix as a namespace — they're a discoverable rename target for forkers but aren't an impediment to reuse (forkers override or alias). Tracked as Task #262.

containers/scripts/resource-policy/apply_caps.py

Lines 67, 82, 85, 87, 89 reference HelixCode-internal paths:

```
# Third-party cli_agents (only HelixCode is ours)
"/cli_agents/helix_code/helix_code/",
# Third-party MCP servers shipped beside HelixCode
"/helix_code/mcp-servers/",
"/HelixLLM/docs/",
```

These are filesystem paths under HelixCode's directory tree hardcoded into a Containers-submodule resource-cap policy script.

Classification: genuine-violation. The containers submodule should not enumerate consuming-project-internal paths.

Action complete (close-out¹⁹, Task #263): refactored. New SKIP_PATH_FRAGMENTS_DEFAULT is universal-only; new --skip-paths-file flag (+ RESOURCE_POLICY_SKIP_PATHS_FILE env var) loads project-specific fragments from a one-per-line file supplied by the consuming project. containers commit 400ccbb. HelixCode-side concrete list at scripts/containers/resource-policy-skip-paths.list.

submodules/llms_verifier (6 files)

- scripts/load_api_keys.sh — same shared utility as containers above. Classification: cosmetic-only. Tracked under Task #262.

- `llm-verifier/capabilities/config_generator.go`
- `llm-verifier/pkg/cliagents/generator.go`
- `llm-verifier/pkg/cliagents/generator_test.go`
- `llm-verifier/pkg/cliagents/helixcode.go`
- `llm-verifier/pkg/cliagents/<sibling>.go`

The `pkg/cliagents/` directory is a per-CLI-agent generator system. `helixcode.go` is the generator FOR HelixCode (siblings would be `claude.go`, `aider.go`, etc.). LLMsVerifier is generic in design; this file is the per-target adapter, named after its target.

Classification: legitimate-cross-project. A per-target generator file is properly named after its target — this is the same pattern as `internal/llm/groq_provider.go` being named after Groq.

Action: none.

github_pages_website (12 files)

Sub-audit (Task #264) results:

File	Refs	Classification
<code>github_pages_website/scripts/load_api_keys.sh</code>	2 (header + docstring)	cosmetic-only → genericised in close-out ¹⁷ (commit <code>c54ff7</code>).
<code>github_pages_website/docs/package.json</code>	2 (description + author)	legitimate-cross-project — npm metadata describes the site, which IS HelixCode's GitHub Pages site.
<code>github_pages_website/docs/start-website.sh</code>	4 (header + log messages)	legitimate-cross-project — bash script that builds + starts HelixCode's docker stack for the website preview. Naming the site in headers + status logs is correct.
<code>github_pages_website/docs/stop-website.sh</code>	1 (header)	legitimate-cross-project — same reasoning.
<code>github_pages_website/docs/test-local.sh</code>	1 (header)	legitimate-cross-project — performance test script for THIS site.
<code>github_pages_website/docs/test-performance.sh</code>	1 (header)	legitimate-cross-project — same.
<code>github_pages_website/docs/test-website.sh</code>	1 (header)	legitimate-cross-project — same.

File	Refs	Classification
github_pages_website/docs/js/main.js	2 (header comment + init log)	legitimate-cross-project — site JS describes the site.
github_pages_website/docs/courses/player.js	refs in content	legitimate-cross-project — course content for HelixCode.
github_pages_website/docs/courses/course-data.js	refs in content	legitimate-cross-project — course data.
github_pages_website/docs/manual/.sync-metadata.json	refs in metadata	legitimate-cross-project — sync metadata referencing the parent project.
github_pages_website/docs/mobile/js/mobile.js	refs in content	legitimate-cross-project — mobile site JS.

Classification: content vs build tooling distinction: - **Build tooling that operates on THIS site** (start-website.sh etc.) is legitimately HelixCode-specific. It's not generic-website tooling; it's THIS site's tooling. Replacing "HelixCode Website Startup Script" with "Site Startup Script" would lose meaning. - **JS/content** describes HelixCode (the subject of the site). Legitimate. - load_api_keys.sh was a cascaded copy of the shared utility — same cosmetic-only treatment as the 3 other copies (Task #262 carried over here in #264).

Action complete. No genuine-violation refactors needed in github_pages_website beyond the load_api_keys.sh fix already landed.

HelixAgent (105 files)

Largest count. HelixAgent is its own meta-project; many of its files cross-reference HelixCode by name. This intersects with Task #254 (HelixAgent's 46 nested own-org submodules — large CONST-051(C) remediation). Per-file review of HelixAgent is properly scoped WITH Task #254, not separately.

Classification: mixed; per-file review deferred until Task #254 underway.

Action: roll into Task #254. No separate work needed here.

helix_qa (1 file)

helix_qa/scripts/load_api_keys.sh — same shared utility.

Classification: cosmetic-only. Tracked under Task #262.

Security (1 file)

security/scripts/load_api_keys.sh — same shared utility.

Classification: cosmetic-only. Tracked under Task #262.

Summary tally

Class	Files	Submodules	Action
legitimate-cross-project	21 (Challenges) + 5 (LLMsVerifier cliagents) + 12 (github_pages_website content) = 38	Challenges, LLMsVerifier, github_pages_website	None.
cosmetic-only	4 × load_api_keys.sh (HelixQA, Security, Containers, LLMsVerifier)	4 submodules	Task #262 — genericise comments.
genuine-violation	1 × apply_caps.py paths-list	containers	Task #263 — refactor to config-file input.
deferred-to-#254	105 (HelixAgent)	HelixAgent	Roll into Task #254 (the larger HelixAgent remediation).

Total decoupling-violation count after this audit: 1 genuine violation (containers/apply_caps.py) + 1 cosmetic batch (4 × load_api_keys.sh) + 1 audit-pending (github_pages_website build scripts).

Audit trail

Date	Reviewer	Round	Notes
2026-05-15	Claude Opus 4.7	round 41 close-out ¹⁶	First pass. Per-file decisions documented. Inline edits deferred to Tasks #262/#263/#264.
2026-05-15	Claude Opus 4.7	round 41 close-out ¹⁷	Task #262 cosmetic-only fix landed (root + helix_qa + Security + Containers). Task #264 sub-audit complete: 1 carry-over load_api_keys.sh cosmetic fix in github_pages_website, 11 other files classified legitimate-cross-project (THIS site's content + tooling).

Sources verified 2026-05-29: internal-governance doc — no third-party-service operator instructions to cross-reference. This is a per-file CONST-051(B) decoupling audit of HelixCode-owned submodules; every reference is to internal paths, scripts (load_api_keys.sh, apply_caps.py), and HelixCode's own submodules. No external service, library, or version pin appears, so there is nothing to WebFetch against vendor docs. Version authority per CLAUDE.md §3.1 (confirmed in-tree): inner module go 1.26, root go 1.25.2. Reviewed against the live tree on this date; no corrections required.